

REST API 설계서

- **팀명:** Rookies_MiniProject2 (소상공인 예약/주문 관리 SaaS)
- **작성자:** 인선
- **API 베이스 URL:** `http://localhost:8080`
- **포맷:** JSON (UTF-8)
- **인증:** JWT Bearer (`Authorization: Bearer <token>`)

목차

1. 공통 규약
2. 인증/권한
3. 에러 응답 규약
4. 엔드포인트 상세
5. 페이징/정렬 규약
6. 변경 이력

1) 공통 규약

- 컬렉션 복수형, 소문자-하이픈 경로 (`/api/stores` , `/api/menus/{menuId}/sold-out` 등).
- 메서드-의미 매핑: GET=조회, POST=생성/액션, PUT/PATCH=수정, DELETE=삭제.
- 서버시간: `LocalDateTime` 직렬화 기준 `yyyy-MM-ddTHH:mm:ss` (UTC 변환 없이 서버 로컬 타임존 그대로 사용 — 운영 환경에서 타임존 고정 여부는 배포 시 확정).
- 모든 쓰기 요청은 CSRF 무관 (STATELESS REST + JWT, 세션/쿠키 미사용).
- 목록 조회는 전부 Spring Data `Page<T>` 표준 응답을 사용 (5장 참고).

2) 인증/권한

- 로그인(`POST /api/auth/login`) 성공 시 `accessToken` (JWT)을 발급하며, 이후 모든 보호된 요청은 `Authorization: Bearer <token>` 헤더가 필요하다.

- 사용자 식별은 클라이언트가 보낸 id가 아니라 토큰에서 직접 추출한다. 커스텀 애너테이션 `@CurrentUser` (`@AuthenticationPrincipal` 기반)가 `UserInfoUserDetails` 에서 로그인한 `User` 엔티티를 바로 꺼내주며, 컨트롤러는 이 값만 신뢰한다.
- 인가는 두 단계로 이루어진다.
 1. **역할 기반:** `@PreAuthorize("hasRole('OWNER')")` / `hasRole('ADMIN')` — 메서드 단위로 역할이 부족하면 403.
 2. **리소스 소유권 검증:** 경로의 `{id}` / `{storeId}` / `{customerId}` 등이 토큰의 로그인 사용자(또는 그 사용자가 소유한 리소스)와 일치하는지 서비스 레이어에서 직접 대조.
- 소유권 불일치 시의 응답은 엔드포인트별로 두 가지 컨벤션이 섞여 있다 (현재 팀 컨벤션 통일 전 상태):
 - **403** `STORE_ACCESS_DENIED` / `USER_ACCESS_DENIED` : 매장/회원이 존재한다는 사실은 알려주되 "당신 소유가 아니다"로 응답 (Store/Menu 도메인 다수)
 - **404** `STORE_NOT_FOUND` : 존재 여부 자체를 노출하지 않음 (`OwnerOrderController` / `SalesController` 의 매장 소유권 검사가 이 방식) — 보안상 더 안전한 선택이라 신규 엔드포인트는 이쪽으로 통일 중
- 비로그인(공개) 엔드포인트: 회원가입, 로그인, 매장 목록/상세 조회(GET), 매장 메뉴 목록 조회(GET), 메뉴 이미지 정적 리소스 조회(`GET /images/**`). 그 외 전부 토큰 필요.

참고: `SecurityConfig` 의 공개 GET 허용 목록은 `/api/stores/**`, `/api/menus/**`, `/images/**` 세 패턴이다. `WebConfig` 가 업로드된 이미지를 `/images/**` 경로로 정적 서빙하므로, 이 패턴이 실제 서빙 경로와 반드시 일치해야 한다 — 과거 이 목록에 `/uploads/**` (디스크 저장 폴더명과 혼동)로 잘못 적혀 있었던 적이 있는데, 당시엔 맨 마지막 `.anyRequest().permitAll()` catch-all 덕분에 우연히 동작했을 뿐 의도한 규칙이 걸린 게 아니었다. 2026-10-02 코드 리뷰에서 발견 및 수정 완료 (6장 변경 이력 참고).

3) 에러 응답 규약

실제 `ErrorResponse` 클래스 기준 응답 형식 (템플릿의 `error.code` / `path` 필드는 우리 프로젝트에는 없음 — 아래가 실제 응답 그대로입니다):

```
{
  "status": 403,
  "message": "본인 소유의 매장만 조회·수정·삭제할 수 있습니다. (id=5)",
  "timestamp": "2026-10-01T10:30:00",
```

```
"errors": null
}
```

@Valid 검증 실패(400) 시에만 `errors` 가 필드별 메시지 맵으로 채워진다:

```
{
  "status": 400,
  "message": "입력값 검증에 실패했습니다.",
  "timestamp": "2026-10-01T10:30:00",
  "errors": { "email": "올바른 이메일 형식이 아닙니다." }
}
```

참고: 응답 본문에 에러 코드 문자열(`VALIDATION_ERROR` 같은)은 노출하지 않지만, 서버 내부적으로는 `ErrorCode` enum으로 에러 종류를 구분한다. 필요하다면 추후 `ErrorResponse` 에 `code` 필드를 추가해 `ErrorCode.name()` 을 내려주는 것도 가능 (현재는 미적용).

내부 `ErrorCode` 전체 목록 (HTTP 상태 매핑)

ErrorCode	HTTP 상태	메시지 템플릿
<code>INVALID_INPUT</code>	400	잘못된 요청입니다: %s
<code>DUPLICATE_EMAIL</code>	409	이미 사용 중인 이메일입니다: %s
<code>INVALID_SIGNUP_ROLE</code>	400	가입 시 선택할 수 없는 권한입니다: %s
<code>INVALID_CREDENTIALS</code>	401	이메일 또는 비밀번호가 일치하지 않습니다.
<code>STORE_NOT_FOUND</code>	404	존재하지 않는 매장입니다. storeId=%s
<code>USER_NOT_FOUND</code>	404	존재하지 않는 회원입니다. userId=%s
<code>USER_ACCESS_DENIED</code>	403	본인 계정만 조회·수정·삭제할 수 있습니다. userId=%s
<code>MENU_ITEM_NOT_FOUND</code>	404	존재하지 않는 메뉴입니다. menuId=%s
<code>MENU_SOLD_OUT</code>	409	품절된 메뉴가 포함되어 있습니다: %s
<code>DUPLICATE_MENU_NAME</code>	409	이미 등록된 메뉴 이름입니다: %s
<code>ORDER_ITEMS_EMPTY</code>	400	주문 항목은 1개 이상이어야 합니다.

ErrorCode	HTTP 상태	메시지 템플릿
ORDER_NOT_FOUND	404	존재하지 않는 주문입니다. orderId=%s
INVALID_ORDER_STATUS	409	올바르지 않은 주문 변경입니다. = %s
INVALID_STORE_MENU_OR_NOT_FOUND	400	주문할 수 없는 메뉴가 포함되어 있습니다. (품질/삭제 등)
STORE_ACCESS_DENIED	403	본인 소유의 매장만 조회·수정·삭제할 수 있습니다. (id=%s)
INVALID_IMAGE_FILE	400	이미지 파일만 업로드할 수 있습니다.
IMAGE_UPLOAD_FAILED	500	이미지 업로드에 실패했습니다: %s

그 외 프레임워크 레벨 공통 처리 (`GlobalExceptionHandler`):

상황	HTTP 상태	메시지
@Valid 검증 실패	400	입력값 검증에 실패했습니다. (+ <code>errors</code> 필드)
요청 본문 파싱 실패/빈 본문	400	요청 본문이 비어있거나 형식이 올바르지 않습니다.
필수 쿼리 파라미터 누락	400	필수 요청 파라미터가 누락되었습니다: {파라미터명}
FK 제약 위반 등 데이터 무결성 충돌	409	연관된 데이터(매장 또는 주문 내역 등)가 존재하여 삭제 또는 수정할 수 없습니다.
지원하지 않는 HTTP 메서드	405	지원하지 않는 HTTP 요청 방식입니다.
@PreAuthorize 역할 부족 (<code>AccessDeniedException</code>)	403	접근 권한이 없습니다
그 외 미처리 예외	500	오류가 발생했습니다. 잠시 기다려주세요.

4) 엔드포인트 상세

UserController (`user/controller/UserController.java`)

참고: 클래스 레벨 `@RequestMapping` 이 없어 메서드마다 전체 경로를 명시한다. 인증 (`/api/auth/*`)과 회원 관리(`/api/users/*`)가 한 클래스에 같이 있음 — 추후

`AuthController` 로 분리하는 것도 리팩터링 후보지만 기능상 문제는 없음(보안 이슈 아님).

POST `/api/auth/signup` — signup

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** 불필요
- **Request Body:**

```
{ "email": "newuser@example.com", "password": "1234", "name": "홍길동", "phone": "010-1234-5678", "role": "USER" }
```

- `role` 은 `USER` 또는 `OWNER` 만 허용 (`ADMIN` 선택 시 400)

Response Type: `ResponseEntity<UserDTO.UserResponse>`

```
{ "id": 4, "email": "newuser@example.com", "name": "홍길동", "phone": "010-1234-5678", "role": "USER" }
```

Status Codes: 201 Created / 400 (검증 실패, `INVALID_SIGNUP_ROLE`) / 409 (`DUPLICATE_EMAIL`)

```
curl -X POST "http://localhost:8080/api/auth/signup" \
  -H "Content-Type: application/json" \
  -d '{"email":"newuser@example.com","password":"1234","name":"홍길동","phone":"010-1234-5678","role":"USER"}'
```

POST `/api/auth/login` — login

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** 불필요
- **Request Body:**

```
{ "email": "owner@rookie.com", "password": "1234" }
```

Response Type: `ResponseEntity<UserDTO.LoginResponse>`

```
{
  "accessToken": "eyJhbGciOi...",
  "user": { "id": 2, "email": "owner@rookie.com", "name": "루키즈사장님", "phone": "010-1111-2222", "role": "OWNER" }
}
```

Status Codes: 200 OK / 401 (`INVALID_CREDENTIALS`)

```
curl -X POST "http://localhost:8080/api/auth/login" \
-H "Content-Type: application/json" \
-d '{"email":"owner@rookie.com","password":"1234"}'
```

GET `/api/users` — getAllUsers

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, **ADMIN 전용**
- **Query Params:** `page` (기본 0), `size` (기본 10), `sort` (기본 `id`)

Response Type: `ResponseEntity<Page<UserDTO.UserResponse>>` **Status Codes:** 200 OK / 401 (토큰 없음) / 403 (ADMIN 아님)

```
curl -X GET "http://localhost:8080/api/users?page=0&size=10" \
-H "Authorization: Bearer <JWT_TOKEN>"
```

GET `/api/users/{id}` — getUser

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `id` (Long) — required

Response Type: `ResponseEntity<UserDTO.UserResponse>` **Status Codes:** 200 OK / 401 / 403 (`USER_ACCESS_DENIED`) / 404 (`USER_NOT_FOUND`)

```
curl -X GET "http://localhost:8080/api/users/3" \
-H "Authorization: Bearer <JWT_TOKEN>"
```

PUT/PATCH `/api/users/{id}` — updateUser

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `id` (Long) — required
- **Request Body** (모든 필드 선택):

```
{ "name": "김손님", "phone": "010-9999-8888", "password": "새비밀번호" }
```

Response Type: `ResponseEntity<UserDTO.UserResponse>` **Status Codes:** 200 OK / 400 (검증 실패) / 401 / 403 (`USER_ACCESS_DENIED`) / 404 (`USER_NOT_FOUND`)

```
curl -X PATCH "http://localhost:8080/api/users/3" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"name": "김손님", "phone": "010-9999-8888"}'
```

DELETE `/api/users/{id}` — deleteUser

- **Consumes:** N/A / **Produces:** N/A
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `id` (Long) — required

Response Type: `ResponseEntity<Void>` (소프트 삭제 — `deletedAt` 기록)

Status Codes: 204 No Content / 401 / 403 (`USER_ACCESS_DENIED`) / 404 (`USER_NOT_FOUND`)

```
curl -X DELETE "http://localhost:8080/api/users/3" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

GET `/api/users/me` — getMyInfo

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT (로그인만 하면 됨, 별도 본인 확인 불필요 — 토큰 자체가 본인)

Response Type: `ResponseEntity<UserDTO.UserResponse>` **Status Codes:** 200 OK / 401 (토큰 없음/만료)

```
curl -X GET "http://localhost:8080/api/users/me" \
-H "Authorization: Bearer <JWT_TOKEN>"
```

StoreController (`user/controller/StoreController.java`)

- **Base Path:** `/api/stores`

GET `` — getAllStores

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** 불필요
- **Query Params:** `category` (선택), `keyword` / `search` (선택, 매장명 검색), `page` (기본 0), `size` (기본 10), `sort` (기본 `id`)

Response Type: `ResponseEntity<Page<StoreDTO.StoreResponse>>`

```
{
  "content": [
    { "id": 1, "name": "루키즈 베이커리", "address": "서울시 강남구 테헤란로 123", "category": "베이커리", "imageUrl": null, "openTime": "08:30:00", "closeTime": "21:00:00", "ownerId": 2, "ownerName": "루키즈사장님" },
  ],
  "totalElements": 1, "totalPages": 1, "number": 0, "size": 10
}
```

Status Codes: 200 OK

```
curl -X GET "http://localhost:8080/api/stores?category=베이커리&keyword=루키즈&page=0&size=10"
```

GET `/{storeId}` — getStore

- **Consumes:** N/A / **Produces:** application/json

- **Auth:** 불필요
- **Path Params:** `storeId` (Long) — required

Response Type: `ResponseEntity<StoreDTO.StoreResponse>` **Status Codes:** 200 OK / 404 (`STORE_NOT_FOUND`)

```
curl -X GET "http://localhost:8080/api/stores/1"
```

POST `` — createStore

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER
- **Request Body:**

```
{ "name": "새 매장", "address": "서울시 ...", "category": "베이커리", "imageUrl": null, "openTime": "09:00:00", "closeTime": "20:00:00" }
```

소유자(`ownerId`)는 요청에 없음 — 토큰의 로그인 사용자로 서버가 자동 지정.

Response Type: `ResponseEntity<StoreDTO.StoreResponse>` **Status Codes:** 201 Created / 400 (검증 실패) / 401 / 403 (OWNER 아님)

```
curl -X POST "http://localhost:8080/api/stores" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"name": "새 매장", "address": "서울시 ...", "category": "베이커리", "openTime": "09:00:00", "closeTime": "20:00:00"}'
```

GET `/owner/{ownerId}` — getStoresByOwner

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인만
- **Path Params:** `ownerId` (Long) — required
- **Query Params:** `page` (기본 0), `size` (기본 10), `sort` (기본 `id`)

Response Type: `ResponseEntity<Page<StoreDTO.StoreResponse>>` **Status Codes:** 200 OK / 401 / 403 (OWNER 아님, 또는 `STORE_ACCESS_DENIED` 로 타인 매장 조회 시도)

```
curl -X GET "http://localhost:8080/api/stores/owner/2?page=0&size=10" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

PUT `/storeId` — updateStore

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인 소유만
- **Path Params:** `storeId` (Long) — required
- **Request Body** (모든 필드 선택):

```
{ "name": "루키즈 베이커리(이전)", "address": null, "category": null, "imageUrl": null, "openTime": "09:00:00", "closeTime": "22:00:00" }
```

Response Type: `ResponseEntity<StoreDTO.StoreResponse>` **Status Codes:** 200 OK / 400 / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`STORE_NOT_FOUND`)

```
curl -X PUT "http://localhost:8080/api/stores/1" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"name": "루키즈 베이커리(이전)", "openTime": "09:00:00", "closeTime": "22:00:00"}'
```

DELETE `/storeId` — deleteStore

- **Consumes:** N/A / **Produces:** N/A
- **Auth:** Bearer JWT, OWNER 본인 소유만
- **Path Params:** `storeId` (Long) — required

Response Type: `ResponseEntity<Void>` (소프트 삭제)

Status Codes: 204 No Content / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`STORE_NOT_FOUND`)

```
curl -X DELETE "http://localhost:8080/api/stores/1" \
-H "Authorization: Bearer <JWT_TOKEN>"
```

MenuController (`menu/controller/MenuController.java`)

참고: 클래스 레벨 `@RequestMapping` 이 없어 메서드마다 전체 경로를 명시한다(보안 이슈 아님).

POST `/api/menus/images` — `uploadMenuImage`

- **Consumes:** multipart/form-data (part명: `image`) / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER

Response Type: `ResponseEntity<String>` (저장된 이미지 URL)

```
"/images/550e8400-e29b-41d4-a716-446655440000.png"
```

Status Codes: 200 OK / 400 (`INVALID_IMAGE_FILE` — 빈 파일 또는 jpeg/png/webp 외 형식) / 401 / 403 (OWNER 아님) / 500 (`IMAGE_UPLOAD_FAILED`)

```
curl -X POST "http://localhost:8080/api/menus/images" \
-H "Authorization: Bearer <JWT_TOKEN>" \
-F "image=@/path/to/bread.png"
```

GET `/api/stores/{storeId}/menus` — `getMenus`

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** 불필요
- **Path Params:** `storeId` (Long) — required
- **Query Params:** `page` (기본 0), `size` (기본 10), `sort` (기본 `id`)

Response Type: `ResponseEntity<Page<MenuItemResponse>>` **Status Codes:** 200 OK

```
curl -X GET "http://localhost:8080/api/stores/1/menus?page=0&size=10"
```

POST `/api/stores/{storeId}/menus` — createMenu

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인 소유 매장만
- **Path Params:** `storeId` (Long) — required
- **Request Body:**

```
{ "name": "소금빵", "price": 3500, "imageUrl": "/images/xxx.png" }
```

Response Type: `ResponseEntity<MenuItemResponse>`

```
{ "id": 10, "name": "소금빵", "price": 3500, "soldOut": false, "imageUrl": "/images/xxx.png" }
```

Status Codes: 201 Created / 400 (검증 실패, 가격 0 이하) / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`STORE_NOT_FOUND`) / 409 (`DUPLICATE_MENU_NAME`)

```
curl -X POST "http://localhost:8080/api/stores/1/menus" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"name":"소금빵","price":3500,"imageUrl":"/images/xxx.png"}'
```

PATCH `/api/menus/{menuId}/sold-out` — toggleSoldOut

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인 소유만
- **Path Params:** `menuId` (Long) — required

Response Type: `ResponseEntity<MenuItemResponse>` **Status Codes:** 200 OK / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`MENU_ITEM_NOT_FOUND`)

```
curl -X PATCH "http://localhost:8080/api/menus/10/sold-out" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

PATCH `/api/menus/{menuId}` — updateMenu

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인 소유만
- **Path Params:** `menuId` (Long) — required
- **Request Body** (모든 필드 선택):

```
{ "name": "소금빵(대)", "price": 4000, "imageUrl": null }
```

Response Type: `ResponseEntity<MenuItemResponse>` **Status Codes:** 200 OK / 400 (가격 0 이하) / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`MENU_ITEM_NOT_FOUND`) / 409 (`DUPLICATE_MENU_NAME`)

```
curl -X PATCH "http://localhost:8080/api/menus/10" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"name": "소금빵(대)", "price": 4000}'
```

DELETE `/api/menus/{menuId}` — deleteMenu

- **Consumes:** N/A / **Produces:** N/A
- **Auth:** Bearer JWT, OWNER 본인 소유만
- **Path Params:** `menuId` (Long) — required

Response Type: `ResponseEntity<Void>` (소프트 삭제)

Status Codes: 204 No Content / 401 / 403 (`STORE_ACCESS_DENIED`) / 404 (`MENU_ITEM_NOT_FOUND`)

```
curl -X DELETE "http://localhost:8080/api/menus/10" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

OrderController (`menu/controller/OrderController.java`)

참고: 클래스 레벨 `@RequestMapping` 이 없음(보안 이슈 아님). 손님의 주문 생성만 담당 — 매장별 주문 목록 조회는 `OwnerOrderController` 로 통합되었다 (아래 참고).

POST `/api/orders` — createOrder

- **Consumes:** application/json / **Produces:** application/json
- **Auth:** Bearer JWT, 로그인(손님)
- **Request Body:**

```
{
  "storeId": 1,
  "items": [ { "menuItemId": 10, "quantity": 2 }, { "menuItemId": 11, "quantity": 1 } ],
  "pickupTime": "2026-10-02T18:30:00",
  "requestNotes": "빵 바삭하게 부탁드립니다"
}
```

Response Type: `ResponseEntity<OrderResponse>`

```
{
  "orderId": 100, "status": "PENDING", "totalAmount": 10500,
  "pickupTime": "2026-10-02T18:30:00",
  "items": [
    { "menuItemId": 10, "menuName": "소금빵", "quantity": 2, "orderPrice": 3500 },
    { "menuItemId": 11, "menuName": "크로아상", "quantity": 1, "orderPrice": 3500 }
  ]
}
```

Status Codes: 201 Created / 400 (검증 실패, 픽업시간 과거,

`INVALID_STORE_MENU_OR_NOT_FOUND`) / 401 / 404 (`STORE_NOT_FOUND`, `USER_NOT_FOUND`) / 409 (`MENU_SOLD_OUT`)

```
curl -X POST "http://localhost:8080/api/orders" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"storeId":1,"items":[{"menuItemId":10,"quantity":
```

```
2}],"pickupTime":"2026-10-02T18:30:00","requestNotes":"빵  
바삭하게 부탁드립니다"}'
```

CustomerOrderController

(`order/controller/CustomerOrderController.java`)

- **Base Path:** `/api/customers/{customerId}/orders`

GET `` — getMyOrders

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `customerId` (Long) — required
- **Query Params:** `page` (기본 0), `size` (기본 10), `sort` (기본 `createdAt`)

Response Type: `ResponseEntity<Page<OrderResponse>>` **Status Codes:** 200 OK / 401 / 403 (`USER_ACCESS_DENIED`)

```
curl -X GET "http://localhost:8080/api/customers/3/orders?page=0&size=10" \  
-H "Authorization: Bearer <JWT_TOKEN>"
```

GET `/_{orderId}` — getMyOrder

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `customerId` (Long), `orderId` (Long) — required

Response Type: `ResponseEntity<OrderResponse>` **Status Codes:** 200 OK / 401 / 403 (`USER_ACCESS_DENIED`) / 404 (`ORDER_NOT_FOUND`)

```
curl -X GET "http://localhost:8080/api/customers/3/orders/100" \  
-H "Authorization: Bearer <JWT_TOKEN>"
```

PATCH `/_{orderId}/cancel` — cancelMyOrder

- **Consumes:** N/A / **Produces:** N/A
- **Auth:** Bearer JWT, 본인만
- **Path Params:** `customerId` (Long), `orderId` (Long) — required

Response Type: `ResponseEntity<Void>` **Status Codes:** 204 No Content / 401 / 403 (`USER_ACCESS_DENIED`) / 404 (`ORDER_NOT_FOUND`) / 409 (`INVALID_ORDER_STATUS` — PENDING이 아닌 주문)

```
curl -X PATCH "http://localhost:8080/api/customers/3/orders/100/cancel" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

OwnerOrderController

(`order/controller/OwnerOrderController.java`)

- **Base Path:** `/api/owner`
- 클래스 레벨 `@PreAuthorize("hasRole('OWNER')")` — 이 컨트롤러의 모든 메서드는 OWNER만 호출 가능.
- 매장 소유권 검증은 `storeRepository.existsByIdAndOwnerIdAndDeletedAtIsNull` 로 수행 — 존재하지 않는 매장이든 타인 소유 매장이든 구분 없이 **404**로 응답한다 (2절의 404 컨벤션 참고).

GET `/stores/{storeId}/orders` — `getOrders`

- **Consumes:** N/A / **Produces:** application/json
- **Auth:** Bearer JWT, OWNER 본인 소유 매장만
- **Path Params:** `storeId` (Long) — required
- **Query Params:** `status` (선택, `PENDING` / `ACCEPTED` / `READY` / `COMPLETED` / `REJECTED` / `CANCELLED`), `page` (기본 0), `size` (기본 10), `sort` (기본 `createdAt`)

Response Type: `ResponseEntity<Page<OwnerOrderResponse>>` — 손님 이름, 주문 생성시각, 요청사항, 품목까지 포함한 사장님용 상세 응답

```
{
  "content": [
    {
```

```

    "orderId": 100, "status": "PENDING", "totalAmount": 1
    0500,
    "pickupTime": "2026-10-02T18:30:00", "createdAt": "20
    26-10-02T17:50:12",
    "customerName": "김손님", "requestNotes": "빵 바삭하게
    부탁드립니다",
    "items": [ { "menuItemId": 10, "menuName": "소금빵",
    "quantity": 2, "orderPrice": 3500 } ]
    },
    ],
    "totalElements": 1, "totalPages": 1, "number": 0, "size":
    10
  }

```

Status Codes: 200 OK / 401 / 403 (OWNER 아님) / 404 (`STORE_NOT_FOUND` — 존재하
지 않거나 본인 소유가 아닌 매장)

```

curl -X GET "http://localhost:8080/api/owner/stores/1/order
s?status=PENDING&page=0&size=10" \
-H "Authorization: Bearer <JWT_TOKEN>"

```

이전 버전(v2.0)에서는 같은 기능이 `menu.controller.OrderController` 의 `GET`
`/api/stores/{storeId}/orders` (403 `STORE_ACCESS_DENIED`, `OrderResponse` 반환)로 별도 존
재했다. 팀 코드 리뷰에서 두 엔드포인트가 중복 구현된 것으로 확인되어 이 엔드포인트
로 통합하고, 구버전 엔드포인트는 삭제했다 (6장 변경 이력 참고).

PATCH `/orders/{orderId}/status` — **updateStatus**

- **Consumes:** application/json / **Produces:** N/A
- **Auth:** Bearer JWT, OWNER 본인 소유 매장 주문만
- **Path Params:** `orderId` (Long) — required
- **Request Body:**

```

{ "status": "ACCEPTED" }

```

허용 값: `ACCEPTED`, `REJECTED`, `READY`, `COMPLETED` (상태 머신 규칙은 도메인 설계서 4.4 참
고)

Response Type: `ResponseEntity<Void>` **Status Codes:** 200 OK / 400 (`status` 누락) / 401 / 403 (OWNER 아님) / 404 (`ORDER_NOT_FOUND` — 존재하지 않거나 본인 매장 주문이 아님) / 409 (`INVALID_ORDER_STATUS`)

```
curl -X PATCH "http://localhost:8080/api/owner/orders/100/status" \
  -H "Authorization: Bearer <JWT_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"status":"ACCEPTED"}'
```

SalesController (`order/controller/SalesController.java`)

- **Base Path:** `/api/owner`
- 클래스 레벨 `@PreAuthorize("hasRole('OWNER')")`, 매장 소유권 검증은 `OwnerOrderController` 와 동일한 404 컨벤션.

GET `/stores/{storeId}/sales/today` — `getTodaySales`

- **Consumes:** N/A / **Produces:** `application/json`
- **Auth:** Bearer JWT, OWNER 본인 소유 매장만
- **Path Params:** `storeId` (Long) — required

Response Type: `ResponseEntity<SalesSummaryResponse>` — 오늘 0시~내일 0시 사이 `COMPLETED` (픽업 완료) 주문만 집계

```
{ "totalSales": 21000, "orderCount": 2 }
```

오늘 완료된 주문이 없으면 `{ "totalSales": 0, "orderCount": 0 }` (서버 쿼리가 `COALESCE(SUM(...), 0)` 처리 — `null` 이 내려오지 않음).

Status Codes: 200 OK / 401 / 403 (OWNER 아님) / 404 (`STORE_NOT_FOUND`)

```
curl -X GET "http://localhost:8080/api/owner/stores/1/sales/today" \
  -H "Authorization: Bearer <JWT_TOKEN>"
```

5)페이징/정렬 규약

- Spring Data `Pageable` 을 그대로 사용 — 커스텀 응답 포맷이 아니라 Spring의 기본 `Page<T>` 직렬화 결과를 그대로 반환한다.
- 쿼리 파라미터: `page` (0-base, 기본 0), `size` (엔드포인트별 `@PageableDefault` 로 기본 값 지정, 대부분 10), `sort` (예: `createdAt,desc`, 다중 정렬도 `sort=a,asc&sort=b,desc` 형태로 가능).
- 응답 예시 (주요 필드만 — 실제로는 `pageable`, `sort`, `first`, `last`, `empty`, `numberOfElements` 등도 함께 내려감):

```
{
  "content": [ { "id": 1 } ],
  "totalElements": 123,
  "totalPages": 13,
  "number": 0,
  "size": 10
}
```

6) 변경 이력

- v2.2 (2026-10-02): 팀 코드 리뷰에서 `SecurityConfig` 의 공개 GET 허용 경로가 `/uploads/**` (실제 서빙 경로는 `WebConfig` 기준 `/images/**`)로 잘못 설정돼 있던 것을 발견, 수정 확인. 2장 인증/권한의 비로그인 엔드포인트 목록에 `/images/**` (메뉴 이미지 정적 리소스)를 누락분으로 추가.
- v2.1 (2026-10-02): 팀 코드 리뷰에서 발견된 중복 엔드포인트 통합 — `menu.controller.OrderController.getOrdersByStore ( GET /api/stores/{storeId}/orders )` 를 삭제하고 `OwnerOrderController.getOrders ( GET /api/owner/stores/{storeId}/orders )` 로 일원화. 응답을 `OrderResponse` → `OwnerOrderResponse` 로, 매장 소유권 실패 응답을 403→404로 통일. 신규 `SalesController` (오늘 매출 조회 API) 문서 추가. 인증/권한 절에 403/404 컨벤션 차이 설명 추가.
- v2.0 (2026-10-01): 도메인별 표 형식(v1.0)에서 컨트롤러별 상세 + cURL 예시 포맷으로 전면 개편. Menu/Order API의 `@CurrentUser` 전환 완료분 반영.
- v1.0 (2026-10-01): 최초 작성 (도메인별 요약 표 + 요청/응답 스펙).